A Faithful Lean Account of the PZG–BEDC Kernel

Unified Theory Working Line

Abstract

We report the portion of the PZG–BEDC dynamic ontology kernel formalized in Lean 4 with mathlib. PZG denotes prime-axis Zeckendorf coding: each prime axis carries a finite Zeckendorf word, and decoding first turns those words into prime exponents and then reconstructs a positive natural number. The checked development covers 40 Lean modules: the finite generation layer, Newman normal forms, prime-axis factorization and Euclid escape, Zeckendorf and PZG bijections, finite golden-ratio algebra, deficit ledgers, Cassini–Fricke trace conservation, finite readings and two-square classifiers, Kleene/Cantor/Gödel barriers for self-coding, cost and hidden-fiber rigidity, critical-line reflection algebra, Euler products, no-zero-line zeta facts, completed zeta functional equations, and the RH ledger equivalence. Every mathematical assertion labeled as machine checked below names an actual Lean declaration under `papers/unified_theory/lean4/UnifiedTheory`. The RH result is an equivalence with mathlib's `RiemannHypothesis`, not a proof of RH; quasicrystal zeta, explicit-formula analysis, positivity bridges, and open leaves such as O-5 and O-6 remain outside the checked-result column. The Lean project is mathlib-based and isolated from the mathlib-free BEDC core; the stated reproducibility surface is the branch `feat/unified-theory-mathlib`, with the full project check given by `lake build`.

1 Introduction

The source theory proposes a dynamic mathematical ontology kernel

$$\mathcal{K}_{\mathrm{PZG}}$$

whose objects are not only static data, but generated histories, standard codes, decoders, normalizers, finite readings, and ledger states. The PZG part of the name is literal: *prime axes* provide the coordinate directions, while *Zeckendorf words* provide the per-axis digit system. In the formalized carrier, a PZG table is a finite-support map

$$p \longmapsto w_p$$

where the support is restricted to primes and each w_p is a legal Zeckendorf representation. Decoding maps each w_p to a natural exponent and then uses unique factorization to recover an element of $\mathbb{N}_{>0}$.

The goal of this Lean line is not to certify the full speculative reach of the source document. It is narrower and more valuable as a logical audit: identify the finite and algebraic claims that can be stated cleanly, prove them in Lean, and keep analytic or self-referential frontier claims outside the checked-result column until they have genuine declarations and proofs. This paper therefore distinguishes three classes.

(i) **Checked theorem or definition**: a declaration in `UnifiedTheory/**.lean` that can be referenced by name.

(ii) **Encoded statement**: a Lean `Prop` used to record a target without pretending that it is proved.

(iii) **Outlook**: mathematical plans or analytic programs that are not yet formalized.

The development is deliberately mathlib-based. It uses mathlib's APIs for natural-number factorization, Zeckendorf representations, golden-ratio identities, pigeonhole arguments, and Fermat's theorem on sums of two squares. This is separate from the mathlib-free BEDC core in the main repository: no file under the BEDC core imports this Lean project, and this project does not alter the BEDC no-mathlib invariant.

2 Formalized Results

This section lists the checked content by layer. Each item gives an informal statement, the precise Lean declaration name, and the reason it captures the corresponding part of the source theory.

2.1 Generation, Mark Exchange, and Histories

Two marks and exchange. The finite basis of the generation layer is the inductive type

`UnifiedTheory.Mark.`

It has exactly two constructors, `zero` and `one`. The exchange operation is

`UnifiedTheory.Mark.sigma : Mark -> Mark,`

with checked facts

`UnifiedTheory.Mark.sigma_involutive`

and

`UnifiedTheory.Mark.sigma_nontrivial.`

Informally, the first theorem says $\sigma(\sigma(x)) = x$ for both marks, and the second records that σ really swaps the two constructors. This is the formal core of the source definitions 1.1–1.2: the minimal binary marker and its nontrivial exchange are not axioms, but a two-constructor inductive type and a case split.

Finite histories. The history carrier is

`UnifiedTheory.MarkHist,`

with constructors `empty` and `ext`. The formal append operation satisfies

`UnifiedTheory.MarkHist.empty_append,`

`UnifiedTheory.MarkHist.append_empty,`

`UnifiedTheory.MarkHist.append_assoc,`

and its length satisfies

`UnifiedTheory.MarkHist.length_append.`

These are the checked version of the finite-history monoid laws in proposition 2.4. The source document describes generation induction and event finiteness as axiomatic principles; the Lean line does not add such axioms. It realizes the finite layer by ordinary inductive recursion and lists.

Events and generation. Events are structures

`UnifiedTheory.Event`

with fields for source history, opcode, argument, and output tag. Event histories are lists, and generation is list extension:

`UnifiedTheory.EventHist.generate.`

The checked statement

`UnifiedTheory.EventHist.generate_length`

says that appending one event increases the list length by one. This is the finite operational shadow of the source idea that a generated event extends a history.

2.2 Uniticity by Abstract Rewriting

The formal rewriting module is generic in a type α and a relation $r : \alpha \to \alpha \to \mathrm{Prop}$. It defines the reflexive-transitive closure

`UnifiedTheory.Rewriting.Star`,

normal forms

`UnifiedTheory.Rewriting.Normal`,

joinability, local confluence, confluence, and termination:

`UnifiedTheory.Rewriting.Joinable`,

`UnifiedTheory.Rewriting.LocallyConfluent`,

`UnifiedTheory.Rewriting.Confluent`,

`UnifiedTheory.Rewriting.Terminates.`

The checked Newman theorem is

`UnifiedTheory.Rewriting.newman_confluent.`

Its type is:

$$\mathrm{Terminates}(r) \to \mathrm{LocallyConfluent}(r) \to \mathrm{Confluent}(r).$$

The normal-form theorem is

`UnifiedTheory.Rewriting.exists_unique_normal_form`,

which states that under the same hypotheses every element has a unique normal form. This exactly formalizes the source theorem 3.2 at the abstract rewriting level. It does not assert that every future normalizer in the theory has already been given a terminating rewrite system; it supplies the reusable criterion.

2.3 Prime Axes and Euclid Escape

The prime-axis state space is

`UnifiedTheory.PrimeExp`,

a finite-support natural exponent vector whose support consists of primes. The declaration

`UnifiedTheory.PrimeExp.equivPNat`

has type

$$\mathbb{N}_{>0} \simeq \texttt{UnifiedTheory.PrimeExp}.$$

This is the formalized prime-axis statement: positive natural numbers are equivalent to finite prime exponent vectors. The proof is a mathlib wrapper around `Nat.factorizationEquiv`, so the paper should not claim that this Lean file rederives the full elementary proof of unique factorization from scratch.

The exponent reading is

`UnifiedTheory.vp`,

and the multiplicative additivity theorem is

`UnifiedTheory.vp_mul`.

It says

$$v_p(mn) = v_p(m) + v_p(n)$$

for $m, n \in \mathbb{N}_{>0}$. This is the checked content behind theorem 4.4's "multiplication is coordinate-wise addition on prime axes" slogan.

Euclid escape is represented by

`UnifiedTheory.euclidEscape`

and proved by

`UnifiedTheory.exists_prime_not_mem`.

For every finite set S of primes, the theorem produces a prime divisor of

$$\prod_{p \in S} p + 1$$

that is not in S. This is the formalized version of theorem 4.5: no finite window of prime axes exhausts the prime directions.

2.4 Zeckendorf Words and the PZG Bijection

On a single axis, the Zeckendorf carrier is

`UnifiedTheory.AxisWord`,

a subtype of lists satisfying mathlib's `List.IsZeckendorfRep`. The equivalence

`UnifiedTheory.AxisWord.equivNat`

has type

$\mathbb{N} \simeq$ `UnifiedTheory.AxisWord`.

The round-trip theorems

`UnifiedTheory.AxisWord.decode_encode`, `UnifiedTheory.AxisWord.encode_decode`

state the existence and uniqueness directions of Zeckendorf representation. This is the checked theorem 5.3, again using mathlib's Zeckendorf API rather than a handwritten proof.

The PZG table carrier is

`UnifiedTheory.PZGTable`.

It is not merely an exponent vector: it keeps the per-prime Zeckendorf word. The axis-level equivalence

`UnifiedTheory.PZGTable.equivPrimeExp`

decodes each axis word to an exponent and preserves prime support. Composing this with prime factorization gives

UnifiedTheory.PZGTable.decode : PZGTable \simeq \Npos .

The checked round trips

UnifiedTheory.PZGTable.decode_encode, UnifiedTheory.PZGTable.encode_decode

are the formal statement of theorem 5.5: PZG tables and positive natural numbers are in bijection.

Normalization is defined canonically:

UnifiedTheory.PZGTable.normAdd z w = encode (decode z * decode w).

The theorem

UnifiedTheory.PZGTable.decode_normAdd

says

$$\mathrm{decode}(\mathrm{normAdd}(z, w)) = \mathrm{decode}(z)\,\mathrm{decode}(w),$$

and

UnifiedTheory.PZGTable.vp_decode_normAdd

says the same thing coordinatewise as prime-exponent addition. This captures theorem 5.6 as a canonical normalization theorem. It is not a proof of a step-by-step carry algorithm or carry termination; it defines the normalized result by the uniqueness of the PZG bijection.

2.5 Golden Double-Sided Algebra

The golden algebra carrier is

UnifiedTheory.PhiInt.

An element is an integer pair representing $a + b\varphi$ with $\varphi^2 = \varphi + 1$. Multiplication, conjugation, and real embeddings are defined by

UnifiedTheory.PhiInt.conj,

UnifiedTheory.PhiInt.toRealPlus,

UnifiedTheory.PhiInt.toRealMinus.

The theorem

UnifiedTheory.PhiInt.fixed_conj_iff

says that a `PhiInt` is fixed by conjugation exactly when its φ-coefficient is zero. This is the algebraic reason the deficit integrality proof works.

The embedding compatibility theorem

UnifiedTheory.PhiInt.toRealMinus_eq_toRealPlus_conj

identifies the shrinking real face with the expanding real face after conjugation, while

UnifiedTheory.PhiInt.toRealPlus_mul

states that the expanding embedding is multiplicative. The Fibonacci decomposition of powers is

UnifiedTheory.PhiInt.phiPow_ab,

which says that the b-coordinate of φ^n is the Fibonacci number F_n.

For axis words, the double-sided reading is

`UnifiedTheory.AxisWord.betaPhi.`

The checked gap identity

`UnifiedTheory.AxisWord.betaPhi_gap`

says that the difference between the expanding and shrinking real readings is $\sqrt{5}$ times the ordinary decoded value. At the PZG-table level, the two real readings are

`UnifiedTheory.PZGTable.lambdaPlus`, `UnifiedTheory.PZGTable.lambdaMinus.`

These declarations formalize definition 6.4 as a finite sum over the finite support of a PZG table.

Carry ledger identities. The finite golden carry identities are checked as

`UnifiedTheory.Golden.gold_carry_plus`, `UnifiedTheory.Golden.gold_carry_minus`,

and the bottom-rule identities as

`UnifiedTheory.Golden.gold_bottom_plus`, `UnifiedTheory.Golden.gold_bottom_minus.`

The exact `PhiInt` version is

`UnifiedTheory.Golden.phiPow_carry.`

These are the formal finite algebra behind proposition 6.21: adjacent Fibonacci carries preserve the double-sided account, while the bottom identity records the unit deficit equation $2\varphi^2 - \varphi^3 = 1$.

Deficit integrality. The normalization deficit is defined by

`UnifiedTheory.deficitPhi.`

The theorem

`UnifiedTheory.deficitPhi_integer`

states that for all $v, w : \mathbb{N}$ there exists $k : \mathbb{Z}$ such that

$$\mathrm{deficitPhi}(v, w) = \mathrm{ofInt}(k).$$

The equivalent conjugation-fixed statement is

`UnifiedTheory.deficitPhi_conj_fixed.`

This is the checked theorem 6.22: the deficit is an integer because its φ-coordinate cancels.

Deficit trichotomy and the shrinking window. The stronger three-value statement is checked unconditionally. The theorem

`UnifiedTheory.abs_betaMinus_le`

proves the shrinking-face window bound

$$|\beta_-(\mathrm{encode}(n))| \le 1/\varphi$$

for every natural number n. The proof uses the gap condition in Zeckendorf words and a finite geometric estimate

`UnifiedTheory.geomBound.`

Combined with deficit integrality, this gives

`UnifiedTheory.deficitTrichotomy_holds`,

the checked theorem 6.25: the integer deficit lies in

$$\{-1, 0, 1\}.$$

This is the first window-length bearing theorem in the Lean development and proves the target

`UnifiedTheory.DeficitTrichotomy`

into a proved theorem.

Beatty-floor deficit and carry conservation. The Beatty deficit module gives an independent floor-arithmetic route to the same three-value phenomenon. It defines the shifted Beatty reading

`UnifiedTheory.S`

and the binary carry deficit

`UnifiedTheory.cDef`.

The theorem

`UnifiedTheory.cDef_mem`

proves directly that `cDef a b` is always -1, 0, or 1. The checked cocycle identity

`UnifiedTheory.carry_conservation`

states

$$c(a, b) + c(a + b, c) = c(a, b + c) + c(b, c),$$

which is the formal carry-conservation law 6.62, or $\delta^2 = 0$, for the Beatty deficit ledger. The bridge from this Beatty reading to shifted Zeckendorf sums is still conditional:

`UnifiedTheory.S_eq_shifted_zeck_of`

requires

`UnifiedTheory.ShiftedZeckendorfBeattyBridge`.

Thus theorem 6.44 is represented as a conditional bridge, not as an unconditional theorem.

n-ary deficit. The list-valued Beatty deficit is

`UnifiedTheory.Cn`.

The decomposition theorem

`UnifiedTheory.Cn_cons`

says that adjoining an entry decomposes the list deficit into one binary deficit plus the tail deficit. The checked trapezoid estimate

`UnifiedTheory.Cn_abs_le`

states

$$|C_n(v_1, \ldots, v_m)| \leq m - 1.$$

This is the symmetric part of theorem 6.60. The sharper asymmetric endpoint count is not claimed here.

Finite Fibonacci core. The finite recurrence

`UnifiedTheory.Golden.finiteW_recurrence`

and Cassini identity

`UnifiedTheory.Golden.cassini`

formalize the finite algebraic core behind the trace-map and generating-function discussion. The traced logarithmic coordinate

`UnifiedTheory.uCoord`

and quadratic form

`UnifiedTheory.qForm`

support the Cassini–Fricke conservation law

`UnifiedTheory.cassini_fricke.`

It states that

$$Q(u_{K+1}, u_K) = 5xy(-1)^{K+1}$$

for

$$u_K = -x\varphi^{K+1} + y\psi^{K+1}.$$

This is the checked theorem 6.36 at the finite trace-map level. These theorems do not formalize the infinite function $W(x, y)$, the quasicrystal zeta object, its Euler-product or continuation theory, or zero transport.

2.6 Finite Readings and Two-Square Classification

The finite-reading pigeonhole theorem is

`UnifiedTheory.Reading.reading_has_fiber.`

Given a finite carrier, a finite class set, and a map into that class set, if the class set has smaller cardinality than the carrier, two distinct elements share the same reading. This proves theorem 8.5 at the level of finite sets: a finite reading with too few values has a nontrivial fiber.

The two-square classifier includes the prime-level theorem

`UnifiedTheory.Reading.prime_sq_add_sq_iff.`

For a prime p, it states

$$(\exists a\, b : \mathbb{N},\ a^2 + b^2 = p) \quad \Longleftrightarrow \quad p \bmod 4 \neq 3.$$

The easy obstruction is proved in the file from the fact that squares modulo 4 are 0 or 1, and the existence direction uses mathlib's theorem for primes that are sums of two squares. This captures the prime-classifier part of chapter 9.

The general classifier is checked:

`UnifiedTheory.Reading.sq_add_sq_iff_factorization.`

It states that a natural number n is a sum of two squares exactly when every prime factor $p \equiv 3 \pmod 4$ appears in the factorization of n with even exponent. This is theorem 9.10 in its ordinary arithmetic form. The proof is a mathlib-based wrapper around the standard sum-of-two-squares factorization theorem; the Lean file records the kernel-facing reading theorem, not an elementary reproof of Fermat's theorem.

2.7 Self-Coding and Diagonal Barriers

The self-code layer contains Cantor diagonal theorems for arbitrary types. The declaration

`UnifiedTheory.SelfCode.no_classifier_surjective`

says that no classifier

$$X \to \mathrm{Set}(X)$$

is surjective. The declaration

`UnifiedTheory.SelfCode.diagonal_not_classified`

exhibits the diagonal property $\{y \mid y \notin \mathrm{classify}(y)\}$ and proves it is not hit by any index. The dual declaration

`UnifiedTheory.SelfCode.no_decoder_injective`

says that no decoder $\mathrm{Set}(X) \to X$ is injective.

These are theorem 16.2–16.3 in their pure diagonal form.

The same layer contains the Kleene fixed-point theorem in mathlib's partial-recursive-code semantics:

`UnifiedTheory.SelfCode.kleene_fixed_point.`

For every computable transformation on `Nat.Partrec.Code`, it produces a code whose partial-function behavior agrees with the transformed code. The bundled chapter statement

`UnifiedTheory.SelfCode.sc1_chapter16_unconditional`

combines Kleene fixed points with the Cantor classifier and decoder barriers. This discharges the machine axiom U at the self-code level by moving to partial-function semantics: no total-termination assumption is needed for theorem 16.1'. It still does not formalize the later internalization tower, same-layer closure predicate, verified substitution evaluator, or Gödel-style arithmeticized residual.

The arithmetic sequence encoder

`UnifiedTheory.SelfCode.godelEncode`

maps a finite list $(l_0, \ldots, l_{m-1})$ to the prime product $\prod_i p_i^{l_i+1}$. Its injectivity theorem

`UnifiedTheory.SelfCode.godelEncode_injective`

is theorem 14.1: two encoded lists with the same Gödel product are equal. This supplies the checked arithmetic infrastructure for self-code separation; the full syntax tower and substitution evaluator remain outside the checked column.

2.8 Dynamics and the Cost Arrow

For event histories, the checked cost theorem is

`UnifiedTheory.EventHist.cost_lt_generate.`

It states that generating one event strictly increases the length of the event history. The theorem

`UnifiedTheory.EventHist.generate_ne`

then says that the generated history is not equal to the original history.

This is the formal content of theorem 18.11–18.12 represented in Lean: irreversibility is expressed as strict growth of the event ledger. It is not the full prime-address dynamics with logarithmic cost over signed exponent states.

The prime-axis logarithmic length is formalized as

`UnifiedTheory.L.`

The checked positivity theorem

`UnifiedTheory.L_pos`

says that every nonzero prime-exponent state has strictly positive logarithmic length. This is theorem 18.7 for the prime-axis state carrier. It supplies the length input for the phase and critical-line ledger, but it does not by itself define a completed heat trace or analytic continuation.

The topology component checks hidden-direction rigidity. The theorem

`UnifiedTheory.Dynamics.hidden_rigidity`

states that a continuous map from a preconnected space into a totally disconnected hidden fiber is constant on that connected input. Its profinite specialization

`UnifiedTheory.Dynamics.profinite_fiber_rigid`

is theorem 20.12: profinite hidden fibers admit no nonconstant continuous path from a preconnected source. This formalizes the rigidity part of the solenoid intuition, not the full solenoid transport or pullback-line program.

2.9 Critical-Line Ledger and Completed Zeta

The phase reading is

`UnifiedTheory.phase,`

with scaling ledger

`UnifiedTheory.Lambda`

and normalized modulus

`UnifiedTheory.normModulus.`

For a nontrivial prime-axis state, the theorem

`UnifiedTheory.unitarity_line_iff`

states that

$$\mathrm{Re}(s) = 1/2 \quad \Longleftrightarrow \quad \mathrm{normModulus}(s, a) = 1.$$

Equivalently, the normalized phase is unitary exactly on the critical line. The reflection

`UnifiedTheory.J`

is defined by $J(s) = 1 - \overline{s}$, and the checked theorem

`UnifiedTheory.J_fixed_iff`

states that its fixed line is exactly $\mathrm{Re}(s) = 1/2$. The scaling ledger changes sign under reflection:

`UnifiedTheory.Lambda_mirror.`

The sign criteria are also checked:

`UnifiedTheory.Lambda_pos_iff`, `UnifiedTheory.Lambda_neg_iff`.

For a nontrivial state, the local scaling ledger is positive exactly on the side $\mathrm{Re}(s) > 1/2$ and negative exactly on the side $\mathrm{Re}(s) < 1/2$. Finally,

`UnifiedTheory.ontic_balance_on_critical_line`

proves that a zero local scaling balance on a nontrivial state forces $\mathrm{Re}(s) = 1/2$. This is theorem 24.8 in its phase-ledger form: the implication from ontic balance to the critical line is unconditional.

The zeta bridge introduces

`UnifiedTheory.ProjectedZero`

for nontrivial zeros of `riemannZeta` and

`UnifiedTheory.OnticZero`

for projected zeros whose local scaling ledger is balanced on every nontrivial prime-axis state. The theorem

`UnifiedTheory.onticZero_re`

proves that every ontic zero lies on the critical line. The bridge theorem

`UnifiedTheory.rh_iff_all_projected_ontic`

states

`RiemannHypothesis` $\iff$ every projected zero is an ontic zero.

This is theorem 24.10 as a ledger restatement of mathlib's `RiemannHypothesis`. It is not a proof of RH, because neither side of the equivalence is discharged unconditionally.

The ordinary zeta side contains the Euler product

`UnifiedTheory.zeta_eulerProduct`,

which states that for $\mathrm{Re}(s) > 1$,

$$\prod_p{}'(1 - p^{-s})^{-1} = \zeta(s).$$

This is theorem 22.3: the analytic avatar of the free prime-axis monoid from theorem 4.4.

The no-zero-line facts are

`UnifiedTheory.zeta_ne_zero_of_one_le_re`, `UnifiedTheory.projectedZero_re_lt_one`.

They say that $\zeta(s) \neq 0$ when $\mathrm{Re}(s) \geq 1$, and that every projected nontrivial zeta zero has $\mathrm{Re}(s) < 1$. This tightens the zero ledger to the critical strip boundary; it does not place those zeros on the critical line.

The completed-zeta interface is also checked. The theorem

`UnifiedTheory.completed_functional_equation`

states

$$\Lambda(1 - s) = \Lambda(s)$$

for mathlib's `completedRiemannZeta`. The explicit `riemannZeta` functional equation with Gamma and cosine factors is

`UnifiedTheory.riemannZeta_explicit_functional_equation`.

The zero-reflection theorem

`UnifiedTheory.completed_zero_reflect`

states that a zero of `completedRiemannZeta` at s gives a zero at $1 - s$. This is one half of the reflected-zero pairing. It gives symmetry about the critical line, not location on the critical line.

2.10 Kernel Components and Ledger States

The kernel aggregate is intentionally thin. The status enum

`UnifiedTheory.Kernel.LedgerStatus`

has four constructors:

`open`, `closed`, `tail`, `semantic`.

The component module supplies aliases such as

`UnifiedTheory.Kernel.History`, `UnifiedTheory.Kernel.ExponentState`,

`UnifiedTheory.Kernel.CodeRegister`, `UnifiedTheory.Kernel.decode`, `UnifiedTheory.Kernel.normalize`.

These names stabilize the formalized slots of $\mathcal{K}_{\mathrm{PZG}}$. They do not constitute a theorem that every component of the full source-theory kernel has been built as one closed record.

Two ledger utilities are checked under `UnifiedTheory.Kernel`. The falsifiability theorem

`UnifiedTheory.Kernel.least_counterexample`

states that a false decidable universal claim over $\mathbb{N}$ has a least counterexample, with all smaller values satisfying the predicate. This is theorem 13.2 as a level-0 finite certificate. The tail-control predicate

`UnifiedTheory.Kernel.TailControlled`

is closed under addition by

`UnifiedTheory.Kernel.tailControlled_add`,

and a residual bounded by a tail budget tending to zero itself tends to zero by

`UnifiedTheory.Kernel.tendsto_zero_of_tailControlled`.

These are the checked tail-additivity and squeeze facts 12.4–12.5.

3 Generated Results (intuition→formalization loop)

The formalization cycle did more than ingest source-document theorems. It generated new checked statements, then fed them back into the paper as mathematical structure discovered by the intuition→formalization→paper loop.

(G1) **Sharp Beatty deficit range.** The theorem

`UnifiedTheory.deficit_trichotomy_sharp`

gives witnesses for all three values: $cDef(0,0) = 0$, $cDef(1,1) = 1$, and $cDef(2,2) = -1$. With `UnifiedTheory.cDef_mem`, the range is exactly $\{-1, 0, 1\}$, so the trichotomy is sharp.

(G2) **Cassini–Fricke nondegeneracy.** The theorem

`UnifiedTheory.cassini_fricke_ne_zero`

proves that the invariant is nonzero exactly when $x \neq 0$ and $y \neq 0$, separating nontrivial Fibonacci-Hamiltonian parameters from the degenerate track $xy = 0$.

(G3) **n-ary ledger composition.** The theorem

`UnifiedTheory.Cn_append`

states $C_n(as{+}{+}bs) = C_n(as)+C_n(bs)+cDef(as.sum, bs.sum)$. It records the coboundary composition law for concatenated finite ledgers.

(G4) **Gödel-code growth.** The theorem

`UnifiedTheory.SelfCode.godelEncode_ge`

proves $2^{\text{length}(l)} \leq \text{godelEncode}(l)$, an exponential lower bound for arithmeticized self-code registers.

(G5) **Critical-side sign test.** The theorems

`UnifiedTheory.Lambda_pos_iff`

`UnifiedTheory.Lambda_neg_iff`

prove $0 < \Lambda_s(a) \iff \text{Re}(s) > 1/2$ and $\Lambda_s(a) < 0 \iff \text{Re}(s) < 1/2$ for nontrivial states. The ledger records the side of the critical line pointwise, without asserting that zeta zeros lie on that line.

4 Scope and Honesty

The checked column has precise boundaries. The Zeckendorf shrinking-window and Beatty-floor routes are

`UnifiedTheory.deficitTrichotomy_holds`

`UnifiedTheory.cDef_mem.`

They are independent mechanisms; the bridge

`UnifiedTheory.S_eq_shifted_zeck_of`

is conditional on

`UnifiedTheory.ShiftedZeckendorfBeattyBridge.`

The n-ary theorem `UnifiedTheory.Cn_abs_le` is the symmetric estimate $|C_n| \leq m - 1$; sharper asymmetric endpoint counts are not claimed. The normalization theorem

`UnifiedTheory.PZGTable.decode_normAdd`

is canonical normalization through the PZG bijection, not termination of a concrete carry-rewrite algorithm.

The zeta results include

`UnifiedTheory.zeta_eulerProduct`

`UnifiedTheory.zeta_ne_zero_of_one_le_re`

`UnifiedTheory.projectedZero_re_lt_one,`

along with completed functional equations and reflected completed-zeta zeros. These facts give prime-axis analyticity in $\text{Re}(s) > 1$, symmetry about the critical line, and exclusion of projected nontrivial zeros from $\text{Re}(s) \geq 1$. They do not prove RH.

The declaration

`UnifiedTheory.rh_iff_all_projected_ontic`

is an equivalence with `RiemannHypothesis`. The theorem `UnifiedTheory.onticZero_re` proves that an already ontic zero lies on the critical line. The missing RH-strength step is still the universal upgrade from projected zeros to ontic zeros; there is no unconditional declaration of type `RiemannHypothesis`.

The checked self-code layer contains Cantor barriers, Kleene fixed points, and the Gödel sequence encoder. It still does not provide a verified instruction set, substitution evaluator, same-layer closure-decider contradiction, or full internalization tower. The checked dynamics layer contains cost growth and profinite hidden-fiber rigidity, not full solenoid transport.

5 Outlook

All items in this section are outside the present checked-result column and are **not yet formalized**. The analytic program still lacks a PZG heat-trace derivation, Mellin/theta bridge, explicit-formula estimates, tail-budget analysis, and a positivity or spectral argument upgrading projected zeros to ontic zeros. RH itself, O-5, and O-6 remain open leaves.

The source theory's quasicrystal zeta object Z_{qc}, its Euler product or continuation theory, natural-boundary claims, diagonal feedback program, and pullback-line claim 6.32 remain outlook. The finite trace-map declarations

`UnifiedTheory.cassini_fricke`

`UnifiedTheory.cassini_fricke_ne_zero`

supply algebraic input for such a program, but they do not build Z_{qc}.

The intended research workflow is correspondence search followed by theorem extraction: prime-axis exponent vectors with Euler products, Beatty–Wythoff and Zeckendorf coding, $\mathbb{Q}(i)$ two-square norms with $\mathbb{Q}(\sqrt{5})$ deficit traces, Hecke–Mahler series with golden word factors, and hidden profinite fibers with rigidity. This manuscript reports only the correspondences backed by actual Lean declarations.

6 Reproducibility

The checked source lives under

`papers/unified_theory/lean4/UnifiedTheory.`

It is a mathlib project, separate from the mathlib-free BEDC core. The full verification command for the Lean project is

```
cd papers/unified_theory/lean4 && lake build.
```

The branch described here is intended to have that check green, with no `sorry` and no `axiom` in the `UnifiedTheory` sources; individual claims reduce to Lean type-checking of the declarations named above.